

FleetOps

A Web-Based Linux Fleet Management System with
Prometheus & Grafana Monitoring

PROJECT REPORT

Submitted by: Divyani Khandait

Roll No. / ID: _____

Department: _____

College / Institution: _____

Guide / Mentor: _____

Date: _____

PROJECT LINKS

GitHub Repository: https://github.com/Divyaaaani/Linux_fleet-3621

Live Demo: _____ (run locally — see Section 11)

1. Executive Summary

FleetOps is a web-based control plane that onboards, monitors, and remotely manages a fleet of Linux machines from a single dashboard, and exposes its telemetry through industry-standard **Prometheus** and **Grafana** monitoring.

Managing many Linux servers by hand does not scale: there is no central view of health, SSH access is painful to configure, machines behind firewalls/NAT are unreachable for pull-based tools, failures go unnoticed, and ad-hoc remote shell access is dangerous. FleetOps solves all five problems with one coherent design — a lightweight **push-based bash agent** on each node and a central **control plane** that stores metrics, raises alerts, dispatches safe (allow-listed) commands, and exports everything to Prometheus for Grafana to visualise.

The system fully satisfies the project brief: it is **web-based**, it **onboards and manages multiple Linux nodes remotely**, and it **collects, stores, and visualises system metrics using Prometheus and Grafana**.

2. Objectives

#	Objective	Delivered
1	Web-based interface to manage a Linux fleet	React dashboard in the browser
2	Onboard multiple Linux nodes remotely	Token-based, single-command agent install
3	Manage nodes remotely	Allow-listed remote command dispatch
4	Collect system metrics	Bash agent reads <code>/proc</code> every 15s
5	Store metrics	SQLite (Turso) time-series tables
6	Visualise metrics	Custom charts + Grafana dashboards
7	Use industry-standard monitoring	Prometheus exporter + Grafana

3. Design Priorities

This project deliberately prioritises **correctness, system design, observability, and an understanding of Linux internals** over UI aesthetics. Each priority is addressed by a concrete design choice:

Priority	How FleetOps addresses it
Correctness	One-time tokens consumed atomically at registration; idempotent metric ingest; a state machine (pending → online → degraded/offline) driven by heartbeat freshness; commands tracked through queued → running → success/failure with results written back. The control plane is the single source of truth.
System design	Clear separation of agent / control plane / presentation tiers. A push-based agent that works behind NAT/firewalls, an allow-listed (never arbitrary) command channel, and a normalised relational schema. Trade-offs are explicit and defensible.
Observability	Native Prometheus exporter in standard exposition format, scraped into Prometheus and visualised in Grafana — plus an automatic alert engine for threshold breaches and missed heartbeats. Every node is measurable end-to-end.
Linux internals	The agent reads directly from the kernel: CPU from <code>/proc/stat</code> , memory from <code>/proc/meminfo</code> , load from <code>/proc/loadavg</code> , uptime from <code>/proc/uptime</code> , disk via <code>df</code> , and process count from <code>/proc</code> . Pure bash + coreutils, no runtime dependencies.

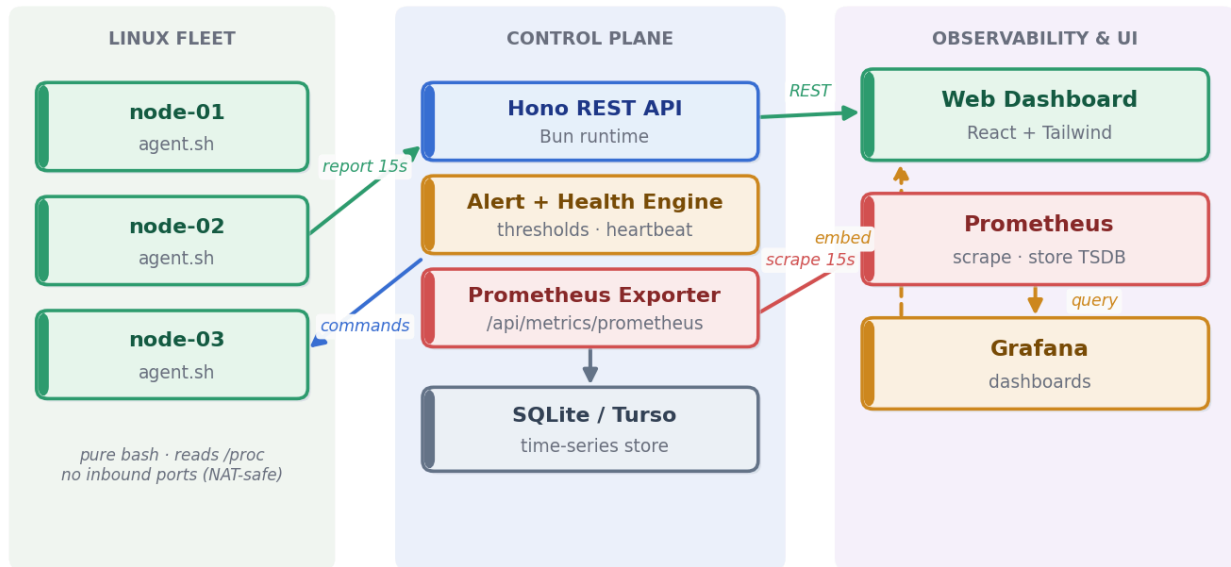
UI aesthetics were intentionally kept functional rather than decorative; the dark operations-centre theme exists only to make dense telemetry readable, not as a design goal in itself.

4. System Architecture

The system has three logical tiers: the **fleet** (agents), the **control plane** (API, storage, alerting, exporter), and the **presentation layer** (web dashboard + Grafana).

FleetOps — System Architecture

push-based agents · central control plane · standard observability



4.1 Components

- **Agent (`agent.sh`)** — a dependency-free bash script that runs on every Linux node. It reads CPU, memory, disk, load average, uptime and process count from `/proc` and `coreutils`, then reports them over HTTP.
- **Control plane** — a **Hono** REST API on the **Bun** runtime, backed by **Drizzle ORM** over **SQLite/Turso**. It registers nodes, ingests telemetry, evaluates alert rules, queues commands, and exposes a Prometheus exporter.
- **Web dashboard** — a **React + Tailwind** single-page app that polls the API every 5 seconds for a live operations-centre feel.
- **Prometheus** — scrapes the control plane's exporter on an interval and stores the metrics as a time series.
- **Grafana** — queries Prometheus and renders dashboards; it is also embedded directly inside the FleetOps UI.

4.2 The key design decision: push, not pull

The agent **pushes** metrics to the control plane and **pulls** queued commands back in the *same* HTTP request. This means a node never needs an open inbound port — only the control plane must be reachable. As a result the system works for machines **behind NAT or firewalls**, which is exactly where traditional pull-based monitoring fails.

Because the agent is push-based, **Prometheus scrapes the control plane's aggregated exporter**, not each individual node. This keeps the Prometheus configuration simple (one target) while still covering the entire fleet.

4.3 Safety by design

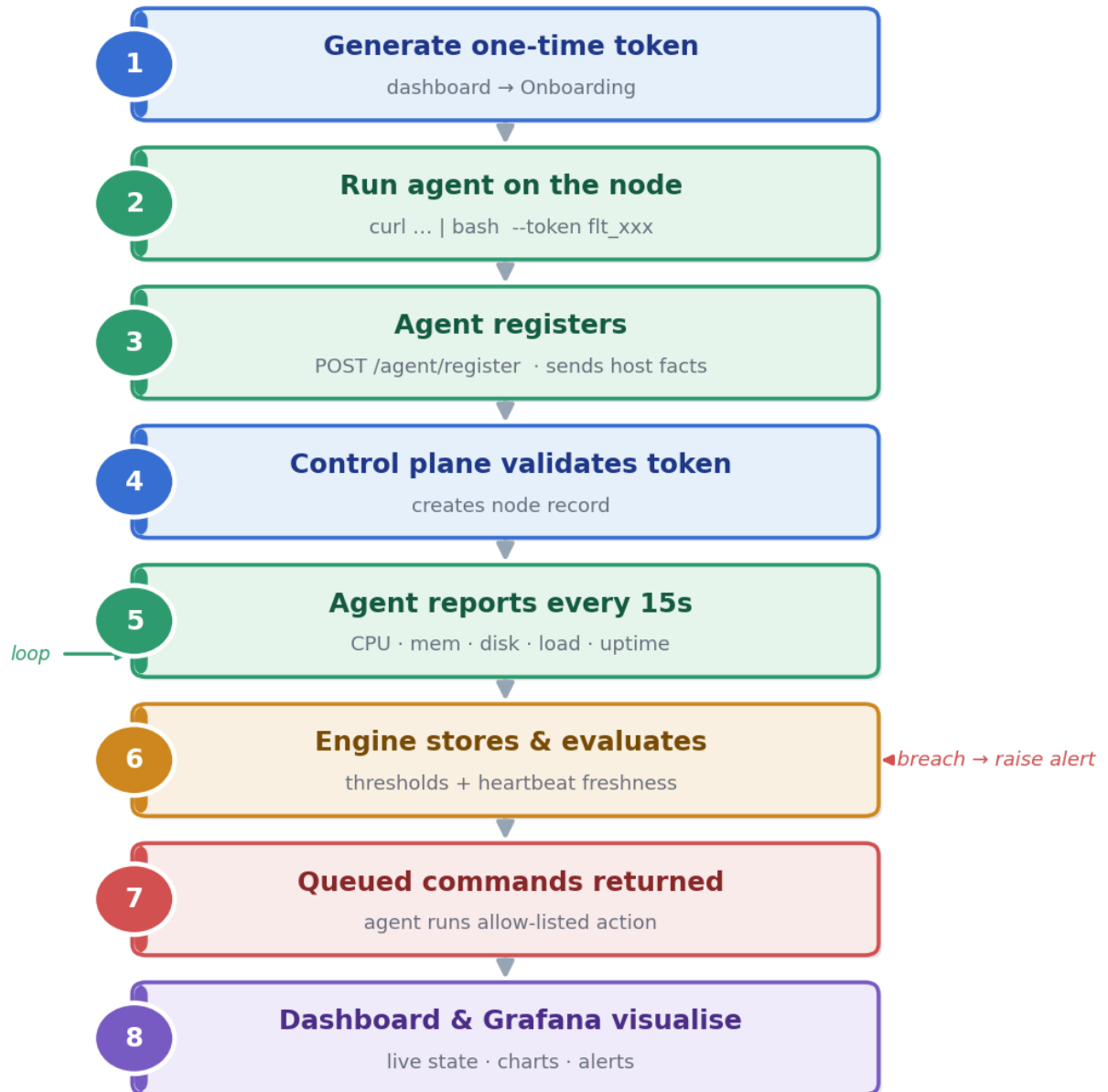
Remote control is restricted to an explicit **allow-list** of actions — refresh metrics, health check, restart agent, restart a named service. The system never executes arbitrary shell sent over the wire, so remote management is safe by construction.

5. Node Lifecycle & Data Flow

The diagram below traces a node from onboarding through continuous reporting and remote control.

Node Lifecycle

Onboard → Report → Control



1. An administrator generates a **one-time token** in the dashboard.
2. The operator runs `agent.sh` on the target node with that token (single `curl ... | bash` command).
3. The agent calls `POST /api/agent/register` with host facts; the control plane validates the token and creates the node record.
4. The agent then loops, calling `POST /api/agent/:id/report` every 15 seconds with fresh metrics.
5. The **alert + health engine** stores each sample and evaluates thresholds and heartbeat freshness.

6. The report response carries any **queued commands**; the agent executes the allow-listed action and reports the result.
 7. The **dashboard and Grafana** visualise live state, time-series charts, and alerts.
-

6. Data Model

The control plane persists five tables in SQLite/Turso:

Table	Purpose
<code>nodes</code>	Registered machines, host facts, current status, last heartbeat
<code>onboarding_tokens</code>	One-time tokens consumed at registration
<code>metrics</code>	Time-series telemetry per node
<code>commands</code>	Queued / running / completed remote actions
<code>alerts</code>	Threshold + heartbeat alerts, with resolution timestamps

A node moves through the states **pending** → **online**, and may become **degraded** (a threshold breach) or **offline** (missed heartbeats), automatically returning to **online** when healthy again.

7. REST API

Method	Route	Purpose
POST	/api/tokens	Generate onboarding token
POST	/api/agent/register	Agent registers with token
POST	/api/agent/:id/report	Heartbeat + metrics; returns queued commands
POST	/api/agent/:id/command-result	Agent reports command result
GET	/api/nodes	Fleet list with latest metrics
GET	/api/nodes/:id/metrics	Time-series (last hour)
POST	/api/nodes/:id/commands	Queue a remote command
GET	/api/alerts	Active + resolved alerts
GET	/api/summary	Fleet health counts
GET	/api/metrics/prometheus	Prometheus exposition format for scraping

8. Prometheus & Grafana Integration

8.1 Exporter

The control plane exposes `GET /api/metrics/prometheus` in the standard Prometheus text exposition format. Each metric is labelled with `node`, `ip`, `env`, and `os` so it can be filtered and aggregated in Grafana.

```
# HELP fleet_cpu_percent CPU utilization percent
# TYPE fleet_cpu_percent gauge
fleet_node_up{node="web-prod-01",ip="10.0.109.188",env="prod",os="Rocky Linux"} 1
fleet_cpu_percent{node="web-prod-01",ip="10.0.109.188",env="prod",os="Rocky Linux"} 43.7
fleet_memory_percent{node="web-prod-01",...} 41.6
fleet_disk_percent{node="web-prod-01",...} 72
fleet_load1{node="web-prod-01",...} 0.16
```


Exported series include `fleet_node_up` , `fleet_cpu_percent` , `fleet_memory_percent` , `fleet_disk_percent` , `fleet_load1` , `fleet_uptime_seconds` , `fleet_processes` , `fleet_alerts_open` , and `fleet_nodes_total` .

8.2 One-command stack

A `docker-compose.yml` brings up the full monitoring stack — the app, **Prometheus**, and **Grafana** — together. Prometheus is pre-configured to scrape the exporter every 15s, and Grafana is **provisioned** automatically with the Prometheus datasource and a ready-made fleet dashboard.

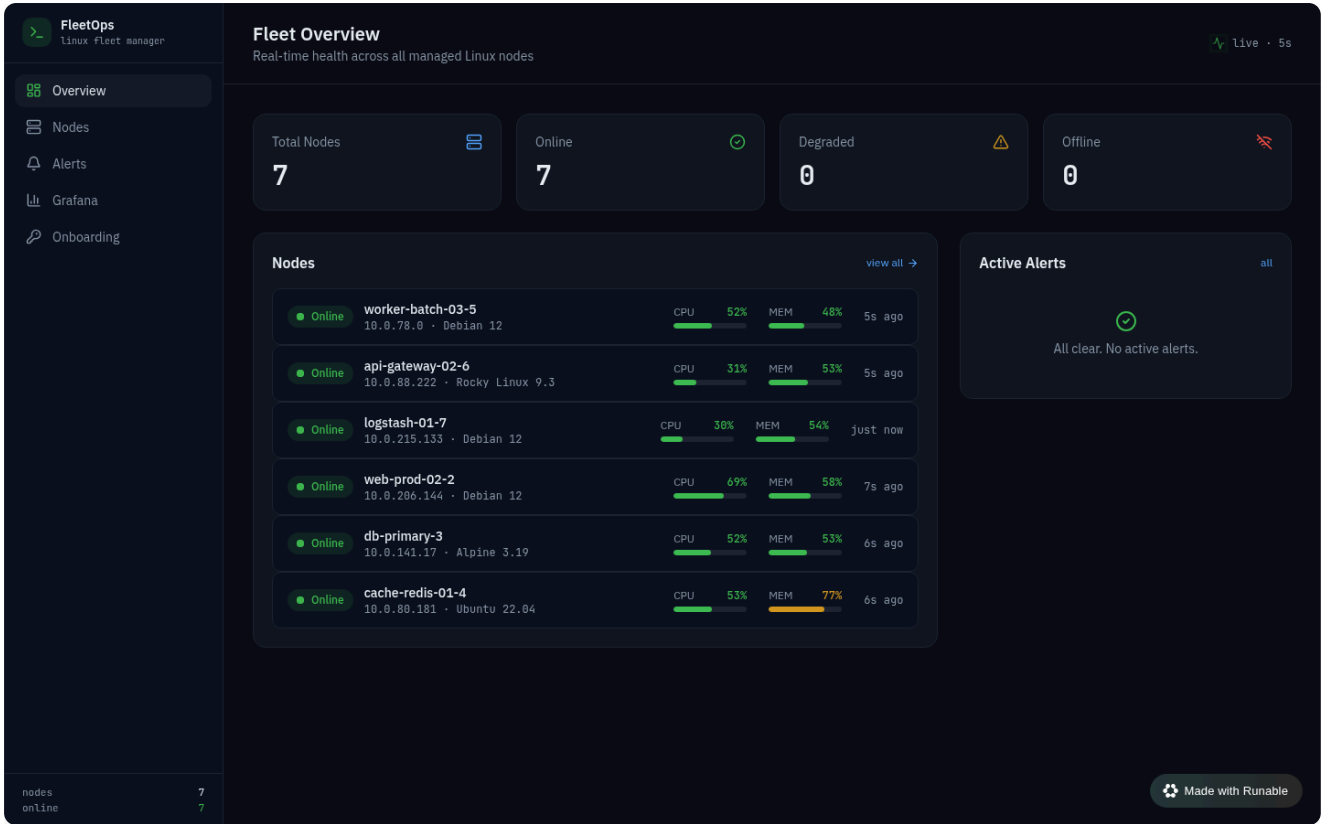
```
docker compose up -d
# App          → http://localhost:4200
# Prometheus   → http://localhost:9090
# Grafana      → http://localhost:3000
```

Grafana is also **embedded directly inside the FleetOps UI** (a dedicated "Grafana" page), so users get the industry-standard dashboards without leaving the application.

9. Implementation Screenshots

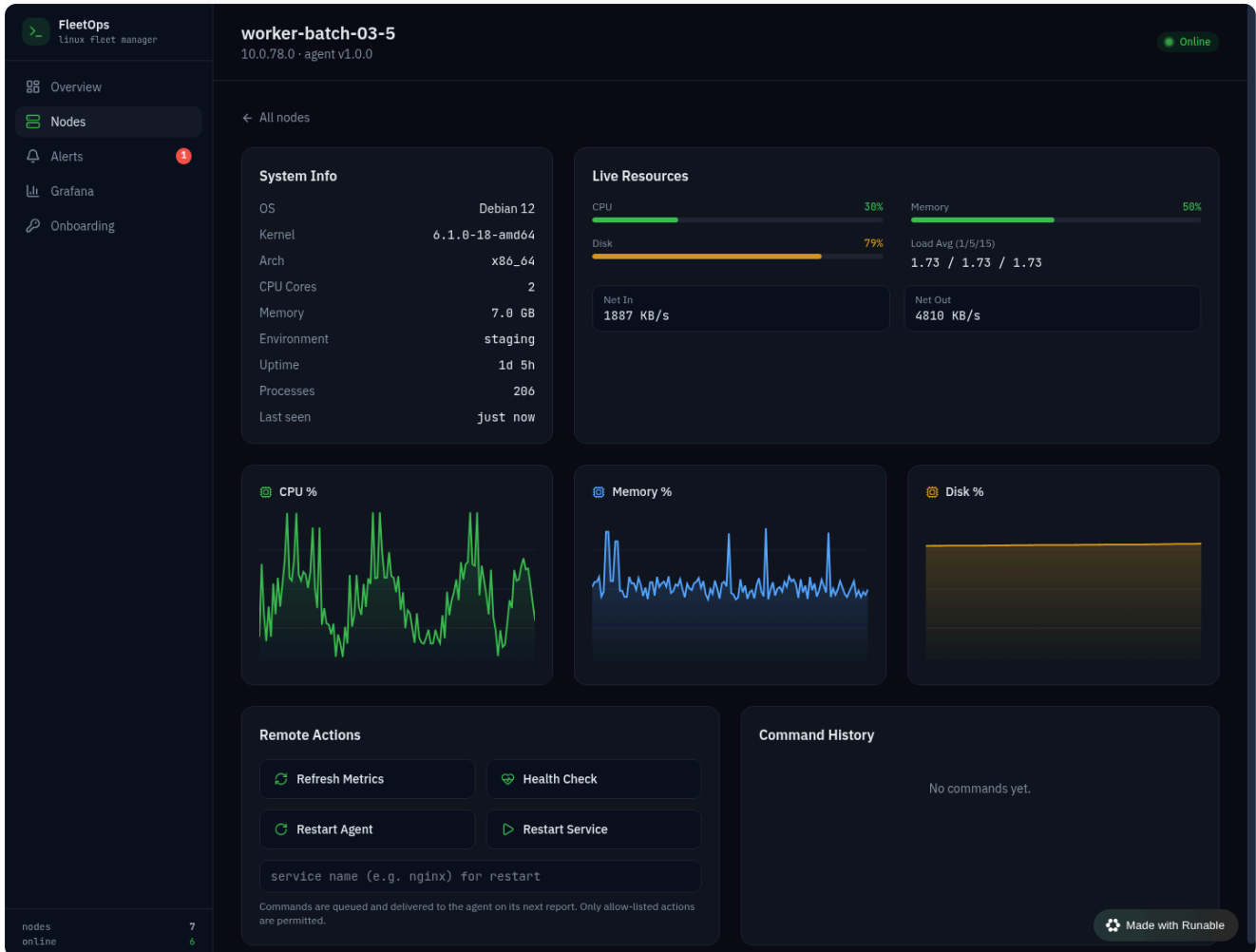
9.1 Fleet Overview

Real-time health cards (total / online / degraded / offline), a live node list with per-node CPU and memory bars, and an active-alerts panel.



9.2 Node Detail

Full system facts, live resource bars, time-series charts for CPU / memory / disk, and a **Remote Actions** panel with command history.



9.3 Fleet — Nodes View

Searchable, filterable table of all managed nodes with status, environment, and latest metrics.

FleetOps

linux fleet manager

Overview

Nodes

Alerts

Grafana

Onboarding

nodes7

online6

Nodes

All managed Linux machines in the fleet

Q Search hostname or IP...

AllOnlineDegradedOffline

STATUS	HOSTNAME	IP	OS	CPU	MEM	DISK	ENV	LAST SEEN
Online	worker-batch-03-5	10.0.78.0	Debian 12	41%	46%	79%	staging	just now
Online	api-gateway-02-6	10.0.88.222	Rocky Linux 9.3	30%	49%	68%	staging	just now
Online	logstash-01-7	10.0.215.133	Debian 12	20%	52%	58%	staging	just now
Online	web-prod-02-2	10.0.206.144	Debian 12	61%	69%	72%	dev	just now
Online	db-primary-3	10.0.141.17	Alpine 3.19	52%	51%	88%	dev	just now
Online	cache-redis-01-4	10.0.80.181	Ubuntu 22.04	52%	65%	63%	dev	just now
Degraded	web-prod-01-1	10.0.109.188	Rocky Linux 9.3	68%	91%	80%	staging	just now

Made with Runnable

FleetOps

linux fleet manager

Overview

Nodes

Alerts

Grafana

Onboarding

nodes7

online6

Onboarding

Generate a token and install the agent on any Linux node

Generate Onboarding Token

Label (optional)

web-server-01

Environment

production

Generate Token

Install Agent

Run this on the target Linux machine. Uses the latest unused token.

curl -fsSL http://localhost:4200/agent.sh | sudo bash -s -- \

--server http://localhost:4200 \

--token flt_e8f050e70f652e8302cf6c1fa9de09f7

No spare machine? Simulate a fleet from your terminal:

SERVER=http://localhost:4200 bash simulate-fleet.sh

Tokens

activeproduction

flt_e8f050e70f652e8302cf6c1fa9de09f7

14m ago

activestaging

flt_2b583b96a55ed5b344a43edbebd090c7

59m ago

activedev

flt_df4d05cc2e78db77bb4fc634923b752

59m ago

activedev

flt_7dba38ed249a19c2232c6bc5edd7b757

11h ago

activedev

flt_5188443abf996df309c72da1afc0e922

11h ago

activedev

flt_f45f1c210f43e6ba001984f7fbb04fa

11h ago

usedworker-batch-03-5staging

flt_659b25b30d01a100963e9d3337fed6b2

12h ago

usedapi-gateway-02-6staging

flt_0f6b3cb69b07bb1be86eec742f7d0303

12h ago

usedlogstash-01-7staging

flt_1d7e025b173284abfdcd4fb8672c1b62

12h ago

Made with Runnable

10. Technology Stack

Layer	Technology
Agent	Bash + coreutils (no dependencies)
Runtime	Bun
API	Hono (REST)
ORM / DB	Drizzle ORM over SQLite (Turso)
Frontend	React 19 + Wouter + Tailwind CSS
Monitoring	Prometheus + Grafana
Orchestration	Docker Compose

11. How to Run

```
# 1. Install dependencies
bun install

# 2. Configure database (.env: DATABASE_URL, DATABASE_AUTH_TOKEN)
cd packages/web && bun run db:push && cd ../..

# 3. Start the control plane + dashboard
bun run dev --port 4200          # → http://localhost:4200

# 4. Onboard a real node
curl -fsSL http://<server>:4200/agent.sh | sudo bash -s -- \
  --server http://<server>:4200 --token flt_xxxxxxx

# 4b. ...or simulate a whole fleet for testing
bash simulate-fleet.sh 7

# 5. Bring up Prometheus + Grafana
docker compose up -d
```

12. Results & Conclusion

FleetOps delivers a complete, working Linux fleet management system that meets every requirement of the brief:

- **Web-based** management of a multi-node Linux fleet from one dashboard.
- **Remote onboarding** in a single command, working even behind NAT/firewalls.
- **Remote management** through a safe, allow-listed command channel.
- **Metric collection** from each node via a lightweight bash agent.
- **Storage** of time-series telemetry in SQLite/Turso.
- **Visualisation** through both a custom real-time dashboard and **industry-standard Prometheus + Grafana**.

The defining engineering choice — a **push-based agent** — is what makes the system practical for real infrastructure, where servers commonly sit behind firewalls. Combined with allow-listed remote control and standard Prometheus/Grafana observability, FleetOps is a small but production-shaped take on distributed Linux fleet management.

Future work

- Role-based access control and audit logging for commands.
- Alerting integrations (email / Slack / PagerDuty) via Prometheus Alertmanager.
- Agent auto-update and rolling command rollout across the fleet.
- Historical metric retention beyond the live window.